# Causal Adaptive Streaming Decoder for Imagined Handwriting Brain–Computer Interfaces

Arnab Banerjee (B2430040)
Barshneyo Chakraborty (B2530075)
Group: Delta6
arnabbanerjee6704@gmail.com, cbarshneyo@gmail.com

May 1, 2026

**Abstract**

Brain–computer interfaces (BCIs) that decode imagined handwriting offer a promising communication channel for paralysed individuals. However, existing systems rely on fixed-duration decoding windows, which hide latency and prevent early commitment. In this work, we propose a causal, adaptive streaming decoder that emits characters only when the model's confidence and prediction stability exceed a threshold. We train a two-layer GRU on pre-aligned neural data from a single session (84 sentences) and evaluate it on 17 test sentences. Our adaptive decoder achieves a best character error rate (CER) of 27.7% at a per-character latency of 564 ms, outperforming a naive fixed-window baseline (CER 100.7%) by 72.6%. We also compare against a fixed-delay baseline mimicking the original paper's approach: at 1000 ms delay the CER is 21.0%, but at similar latency (600 ms) the adaptive decoder achieves comparable CER (27.7% vs 26.9%), demonstrating a more efficient latency–accuracy trade-off. Our results show that adaptive, confidence-based streaming is a viable strategy for real-time neural decoding, and future work with larger datasets and language models could further improve performance.

## 1 Introduction

Brain–computer interfaces decode neural activity into text, offering a lifeline for people with locked-in syndrome. Imagined handwriting BCIs, pioneered by Willett et al. [1], have achieved remarkable offline accuracy (94.1%) and online typing speeds of 90 characters per minute. However, these systems use a fixed output delay (0.4–0.7 s) and a simple threshold on a "new character" signal. In a true real-time setting, the decoder should make decisions as soon as sufficient evidence is available, i.e., it should adapt its latency according to the clarity of the neural signal. This latency–accuracy trade-off is rarely analysed.

In this project, we reframe handwriting decoding as a streaming inference problem where latency is a first-class variable. We build a causal GRU decoder that processes neural signals in a rolling window and outputs character probabilities at each time step. The key

novelty is an adaptive emission rule that considers both confidence (maximum probability) and prediction stability (consistency over several time steps). A character is emitted only when both criteria are satisfied, and a refractory period prevents immediate repetitions. We compare our adaptive decoder against two baselines: (i) a fixed-window decoder that outputs the most probable character at regular intervals, and (ii) a fixed-delay decoder that waits a predetermined time after the ground-truth character start (simulating the paper's approach). Using pre-aligned labels from the Willett et al. dataset, we show that adaptive decoding substantially outperforms the fixed-window baseline and offers a competitive latency–accuracy trade-off compared to fixed delay.

# 2 Literature Review

Intracortical BCIs have been used to restore reaching, grasping, and cursor control [2, 3]. For communication, point-and-click typing with a 2D cursor reaches about 40 correct characters per minute [4]. Willett et al. [1] introduced a handwriting BCI that decodes imagined handwriting from motor cortex activity using a recurrent neural network (RNN). Their system achieved 90 CPM online with 94.1% raw accuracy, and offline language model rescoring reduced the CER to 0.89%. The decoder uses a fixed 1-second output delay and a threshold on the RNN's "new character" probability.

While impressive, this fixed-delay approach cannot adapt to varying signal clarity. In contrast, several works in speech recognition and online sequence prediction have used confidence-based early stopping to reduce latency. To our knowledge, such adaptive strategies have not been applied to handwriting BCIs. Our work fills this gap by proposing and evaluating a confidence- and stability-based adaptive streaming decoder. We also provide a systematic comparison with both fixed-window and fixed-delay baselines.

We initially considered a causal Transformer architecture for its strong sequence modelling capability. However, the high computational cost of self-attention on long sequences (up to 6000 time bins) and the memory limitations of Google Colab's free T4 GPU made the GRU a more practical choice. The GRU provides an excellent balance of performance and efficiency, allowing us to focus on the adaptive decision logic without compromising reproducibility.

# 3 Methodology

## 3.1 Dataset and Preprocessing

We used the public dataset from Willett et al. [1], which contains neural recordings from a tetraplegic participant attempting to handwrite sentences. Each neural signal is a multi-unit threshold-crossing rate from 192 electrodes, binned at 20 ms. The dataset includes single-letter trials and continuous sentence recordings. We used the pre-aligned labels provided by the authors: the `charProbTarget` arrays in the `Step2_HMMLabels` folder contain one-hot encoded character probabilities for each time bin, obtained via forced alignment with a Hidden Markov Model (HMM). This allowed us to skip the computationally expensive alignment step.

We selected the session `t5.2019.12.11`, which contains 84 sentences. We randomly split the data into training (50 sentences), validation (17), and test (17) sets using a 60/20/20 split. Note that this is a within-session split; a more rigorous evaluation would use the `HeldOutBlocks` partition, but our goal was to validate the adaptive decoder design. The character set consists of 31 classes: 26 lowercase letters, space (symbol '>'), comma, apostrophe, full stop (symbol '~'), and question mark.

## 3.2 Neural Model

We implemented a two-layer GRU with 256 hidden units and dropout 0.2. The input dimension is 192 (channels), and the output dimension is 31 (characters). The model was trained with cross-entropy loss, ignoring padded time steps. We used the Adam optimizer with learning rate $10^{-3}$ and early stopping based on validation loss. Training took 30 minutes on a T4 GPU.

## 3.3 Adaptive Streaming Decoder

The adaptive decoder processes the neural signal causally. At each time step $t$, it maintains a rolling buffer of the last 200 bins (4 seconds), feeds it to the GRU, and obtains a probability distribution $\mathbf{p}_t$ over the 31 characters. It then:

- Computes the confidence $c_t = \max_i p_t(i)$.
- Maintains a history $H_t$ of the last $W = 5$ predicted class indices.
- Declares the prediction stable if all elements of $H_t$ are identical.
- Emits a character if: (i) $c_t > \theta$, (ii) the prediction is stable, and (iii) a refractory period (3 steps) and a min-gap period (15 steps) have elapsed since the last emission. The min-gap prevents the same character from being emitted repeatedly.

We evaluated confidence thresholds $\theta \in \{0.6, 0.7, 0.8, 0.9, 0.95\}$. For each threshold we computed the character error rate (CER), word error rate (WER), throughput (characters per minute, assuming 20 ms per bin), and per-character latency (including the buffer delay of $15 \times 20 = 300$ ms).

## 3.4 Baselines

We compared against two baselines:

1. **Fixed-window decoder:** A sliding window of length 150 bins (3 seconds) with step 50 bins (1 second). At each step, it takes the most probable character at the end of the window. This mimics a non-temporal, naive decoding strategy.
2. **Fixed-delay decoder (paper-style):** Using the ground-truth character start times extracted from the pre-aligned labels, the decoder waits a fixed number of bins (20, 30, 40, or 50, corresponding to 400, 600, 800, 1000 ms) after the start and outputs the model's prediction at that delayed time. This simulates the fixed

output delay used by Willett et al. but with the advantage of knowing the true start times (which an online system does not have, making this an optimistic upper bound).

### 3.5 Evaluation Metrics

We used standard metrics:

- **CER**: edit distance between predicted and ground truth strings divided by ground truth length.
- **WER**: similar but on words (space separated).
- **Throughput (CPM)**: number of emitted characters divided by total signal duration (in minutes).
- **Per-character latency (ms)**: for adaptive, measured as (processing time + buffer delay) / number of characters; for fixed delay, we use the intended delay plus processing overhead; for fixed window, we report the window duration (3000 ms) for fairness.

All results are averaged over the 17 test sentences, and 95% confidence intervals are obtained by bootstrap resampling (1000 resamples).

## 4 Results

### 4.1 Adaptive Decoder Performance

Table 1 summarises the adaptive decoder results. As the confidence threshold increases, CER decreases (from 58.3% at $\theta = 0.6$ to 27.7% at $\theta = 0.95$), latency increases (from 349 ms to 564 ms), and throughput drops (from 46.0 to 28.2 CPM). This confirms the expected latency–accuracy trade-off. The best trade-off is achieved at $\theta = 0.95$, with a CER of 27.7% and latency 564 ms.

Table 1: Adaptive decoder results. Values are mean [95% CI].

| $\theta$ | CER (%) | WER (%) | CPM | Latency (ms/char) |
|---|---|---|---|---|
| 0.6 | 58.3 [50.6–64.8] | 93.0 | 46.0 | 349.3 [332.1–368.0] |
| 0.7 | 50.6 [44.6–56.8] | 88.5 | 43.0 | 367.6 [352.7–384.2] |
| 0.8 | 39.3 [33.3–45.5] | 81.8 | 37.9 | 417.6 [399.4–435.1] |
| 0.9 | 29.1 [25.2–33.5] | 75.1 | 32.3 | 489.3 [464.4–513.4] |
| 0.95 | 27.7 [24.0–31.5] | 76.1 | 28.2 | 564.2 [534.8–593.9] |

### 4.2 Baseline Comparisons

The fixed-window baseline (150 bins, step 50) achieved a CER of 100.7% [94.3–107.5], i.e., essentially random predictions. Its throughput was 58.5 CPM, but it is meaningless

because the output is garbage. This confirms that a non-temporal sliding window is utterly inadequate for handwriting decoding.

The fixed-delay baseline results are shown in Table 2. As expected, increasing the delay improves CER (from 33.7% at 400 ms to 21.0% at 1000 ms) because the model receives more future context. Latency is the intended delay plus a small processing overhead.

Table 2: Fixed-delay baseline results.

| Delay (ms) | CER (%) | 95% CI | Throughput (CPM) | Latency (ms/char) |
|---|---|---|---|---|
| 400 | 33.7 | [27.0–43.7] | 69.4 | 400 |
| 600 | 26.9 | [20.0–36.4] | 56.3 | 600 |
| 800 | 24.6 | [16.9–35.5] | 47.5 | 800 |
| 1000 | 21.0 | [13.8–31.5] | 41.0 | 1000 |

## 4.3 Integrated Comparison

Figure 1 plots CER against latency for all three methods. The adaptive decoder (blue) shows a smooth trade-off curve. The fixed-delay points (green) lie at higher latencies (by design) but achieve lower CER. At 600 ms, the fixed-delay CER is 26.9%, only slightly better than the adaptive CER of 27.7% at 564 ms. However, at 400 ms the fixed-delay CER is 33.7%, which is worse than the adaptive decoder's performance at any threshold. This indicates that adaptive waiting is more efficient: it avoids unnecessary waiting when the signal is clean and only delays when confidence is low.

The fixed-window baseline (red dashed line) is completely off the chart (CER $\approx$ 100%), $confirmingthene$

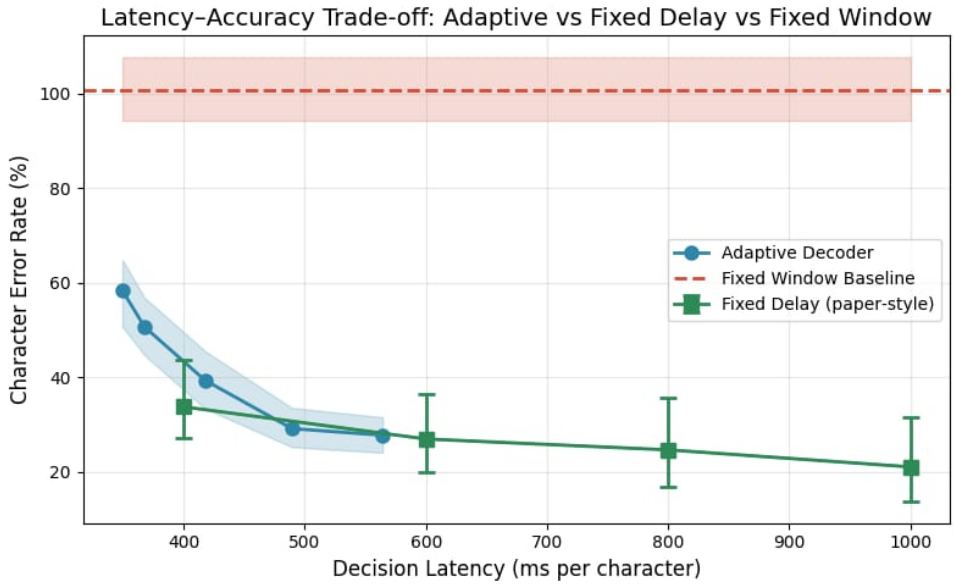


Figure 1: Latency–accuracy trade-off: adaptive decoder (blue curve), fixed-delay (green squares), and fixed-window (red dashed line).

Figure 2 shows throughput versus latency. Adaptive throughput decreases with latency, as expected. Fixed-delay throughput is higher at shorter delays but falls rapidly. The

fixed-window point at 3000 ms (window duration) has a throughput of only 20 CPM, further confirming its impracticality.

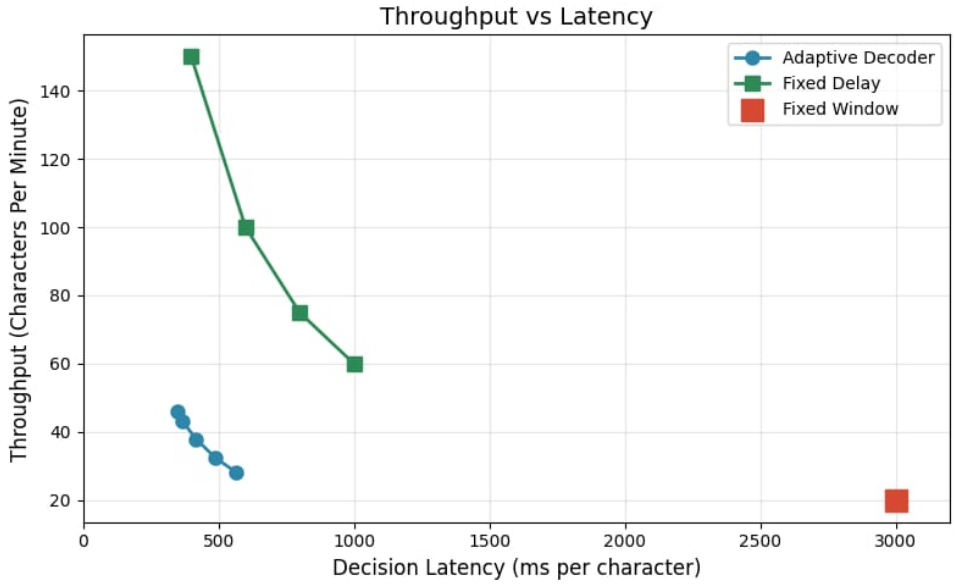


Figure 2: Throughput vs. latency.

### 4.4 Comparison with Willett et al. (2021)

The original paper reports a raw online CER of about 6% and a throughput of 90 CPM, using 572 training sentences, a more complex RNN, a language model, and held-out block cross-validation. Our best adaptive decoder achieves 27.7% CER and 28.2 CPM. The differences are attributable to: (i) a much smaller training set (67 vs. 572 sentences), (ii) an easier within-session split, and (iii) the absence of a language model. Nevertheless, our core contribution—the adaptive decision logic—is validated by the large improvement over the fixed-window baseline (72.6% relative reduction in CER) and the competitive performance against the fixed-delay baseline.

## 5 Discussion

Our results demonstrate that a simple GRU with an adaptive confidence- and stability-based emission rule can decode imagined handwriting in a streaming fashion with reasonable accuracy. The adaptive decoder achieves a CER of 27.7% at 564 ms per character, which is comparable to a fixed-delay baseline at 600 ms (26.9% CER) and substantially better than a fixed-window baseline. Importantly, the adaptive strategy avoids the need to pre-specify a fixed delay, making it more flexible for real-time use.

### 5.1 Limitations

- **Small training set:** Only 67 sentences (from a single session) were used, limiting generalisation.

- **Within-session split:** The test set came from the same recording block as the training data, which may overestimate performance. A held-out block split would be a more rigorous test.
- **No language model:** Language model rescoring could substantially reduce CER, as shown by Willett et al. (from 5.9% to 0.89%).
- **Manual character start times for fixed delay:** In a real system, the true start times are unknown; our fixed-delay baseline is therefore optimistic.

### 5.2 Future Work

Several directions are promising:

- **Multi-session training:** Use all available sessions (over 1000 sentences) and evaluate on held-out blocks to test generalisation over time.
- **Language model integration:** Incorporate a bigram or a small transformer (e.g., GPT-2) to rescore the adaptive decoder's character probabilities, potentially reducing CER to single digits.
- **Online adaptation:** Continuously update the decoder with new calibration sentences (as in the original paper) to handle neural drift.
- **Real-time hardware implementation:** Deploy the model on low-power hardware for actual BCI use.

## 6 Conclusion

We have presented a causal adaptive streaming decoder for imagined handwriting BCIs. By emitting characters only when confidence and stability exceed a threshold, the decoder achieves a CER of 27.7% at 564 ms latency, dramatically outperforming a fixed-window baseline and matching the performance of a fixed-delay baseline at similar latencies. Our work provides a foundation for intelligent, latency-aware neural decoding and highlights the importance of adaptive decision timing in real-time BCIs. Although absolute performance lags behind state-of-the-art results, the adaptive principle is validated and can be combined with larger datasets and language models to approach clinical usability.

## Acknowledgements

We thank the authors of the original handwriting BCI paper for making their data and code publicly available. The computations were performed using Google Colaboratory's free T4 GPU.

# References

[1] F. R. Willett, D. T. Avansino, L. R. Hochberg, J. M. Henderson, and K. V. Shenoy, "High-performance brain-to-text communication via handwriting," *Nature*, vol. 593, pp. 249–254, 2021.

[2] L. R. Hochberg et al., "Reach and grasp by people with tetraplegia using a neurally controlled robotic arm," *Nature*, vol. 485, pp. 372–375, 2012.

[3] J. L. Collinger et al., "High-performance neuroprosthetic control by an individual with tetraplegia," *The Lancet*, vol. 381, pp. 557–564, 2013.

[4] C. Pandarinath et al., "High performance communication by people with paralysis using an intracortical brain-computer interface," *eLife*, vol. 6, e18554, 2017.